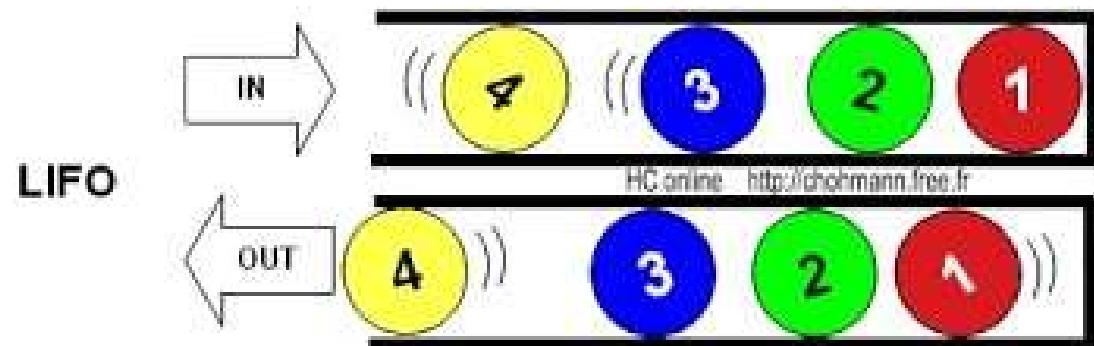


Data Structures (CS2103)

Stack & Queue

Prepared By – Ashish K Layek
CST, IEST Shibpur

Stacks

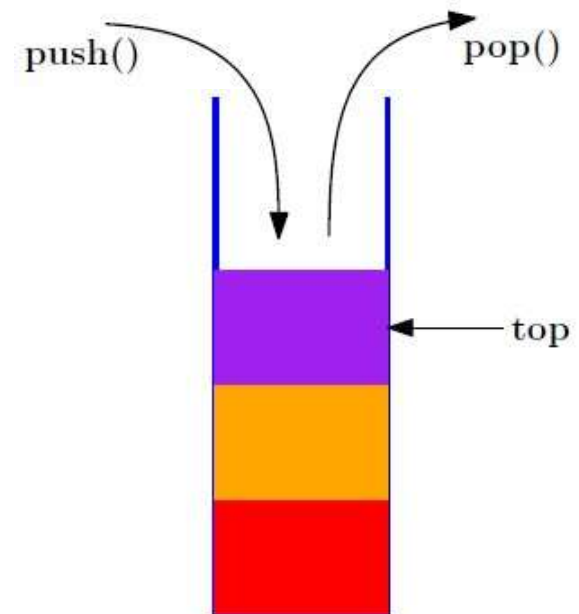


Input sequence 1, 2, 3, 4 $\neq$ Output sequence 4, 3, 2, 1

Overview

Definition: A stack is an ordered list in which all insertions and deletions are made at one end, called the top.

➤ Last-In-First-Out Data Structure



The Stack ADT

- ❑ The Stack ADT stores arbitrary objects
 - ❑ Insertions and deletions follow the last-in first-out scheme
 - ❑ Think of a spring-loaded plate dispenser
 - ❑ Main stack operations:
 - **push(object)**: inserts an element
 - **object pop()**: removes the last inserted element
 - ❑ Auxiliary stack operations:
 - **object top()**: returns the last inserted element without removing it
 - **integer size()**: returns the number of elements stored
 - **boolean empty()**: indicates whether no elements are stored
-

Applications of Stacks

❑ Direct applications

- Page-visited history in a Web browser
- Undo sequence in a text editor
- Chain of method calls in the C/C++ run-time system

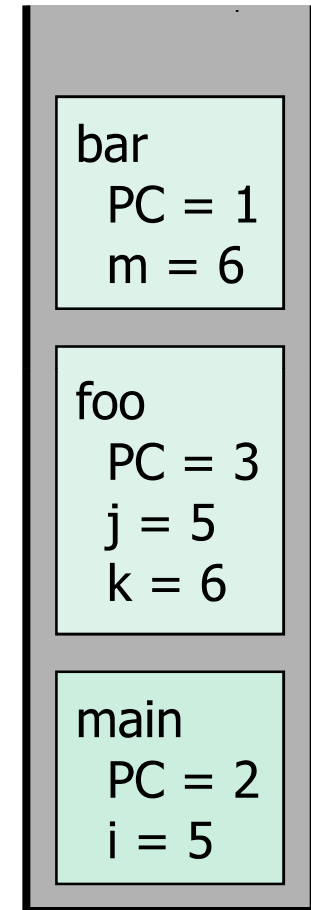
❑ Indirect applications

- Auxiliary data structure for algorithms
 - Component of other data structures
-

Example: C Run-Time Stack

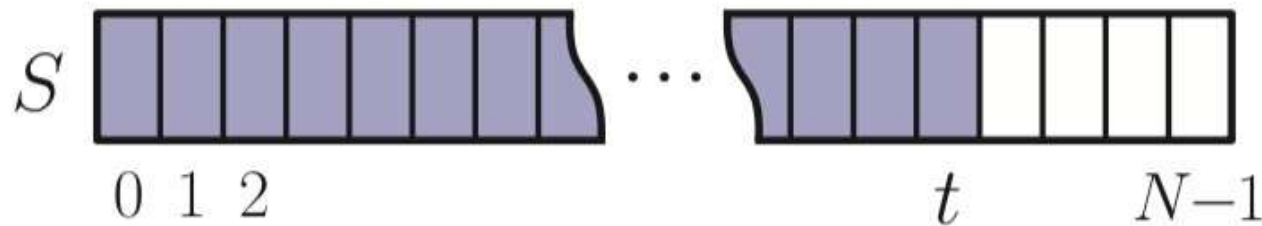
- ❑ The C++ run-time system keeps track of the chain of active functions with a stack
- ❑ When a function is called, the system pushes on the stack a frame containing
 - Local variables and return value
 - Program counter, keeping track of the statement being executed
- ❑ When the function ends, its frame is popped from the stack and control is passed to the function on top of the stack
- ❑ Allows for recursion
- ❑ PC: Program Counter

```
main() {  
    int i = 5;  
    foo(i);  
}  
  
foo(int j) {  
    int k;  
    k = j+1;  
    bar(k);  
}  
  
bar(int m) {  
    ...  
}
```



Example Implementation: Array-based Stack

- ❑ A simple way of implementing the Stack ADT uses an array
- ❑ We add elements from left to right
- ❑ A variable keeps track of the index of the top element



Example Implementation: Array-based Stack

- ❑ A simple way of implementing the Stack ADT
- ❑ Add elements from left to right
- ❑ A variable keeps track of the index of the top element
- ❑ The array storing the stack elements may become full
 - A push operation will then throw an Error message
 - Limitation of the array-based implementation
 - Not intrinsic to the Stack ADT

Algorithm size():

return $t + 1$

Algorithm empty():

return $(t < 0)$

Algorithm top():

if empty() **then**

 throw StackEmpty exception

return $S[t]$

Algorithm push(e):

if size() = N **then**

 throw StackFull exception

$t \leftarrow t + 1$

$S[t] \leftarrow e$

Algorithm pop():

if empty() **then**

 throw StackEmpty exception

$t \leftarrow t - 1$

Performance and Limitations

❑ Performance

- Let n be the number of elements in the stack
- The space used is $O(n)$
- Each operation runs in time $O(1)$

❑ Limitations

- The maximum size of the stack must be defined a priori and cannot be changed
- Trying to push a new element into a full stack causes an implementation-specific exception

Solution: Linked-allocation based Stack (discuss later)

Stack Applications: Parentheses Matching

- ❑ Each "(", "{", or "[" must be paired with a matching ")", "}", or "]"
 - correct: () (()) { ([()]) }
 - correct: ((() (()) { ([()]) }
 - incorrect:) (()) { ([()]) }
 - incorrect: ({ []) }
 - incorrect: (

 - ❑ Good Programmer
 - Someone who thinks that stack is a good data structure for the above task
-

Parentheses Matching Algorithm

```
valid = TRUE; /*Assume the string is valid */
S = the empty stack;
while (the entire string is not read) {
    read the next symbol (symb) of the string;
    if (symb == '(' || symb == '{' || symb == '[')
        push(S, symb)
    if (symb == ')' || symb == '}' || symb == ']')
        if empty(S) {
            valid = false; break;
        } else {
            elem = pop(S);
            if(elem is not matching opener of symb) {
                valid = false; break;
            } /* end if */
        } /* end else */
    } /* end while */
if ( valid && !empty(S)) {
    valid = false;
}
```

Depending upon the value of valid flag, it can be concluded if the parenthesis matched or not.

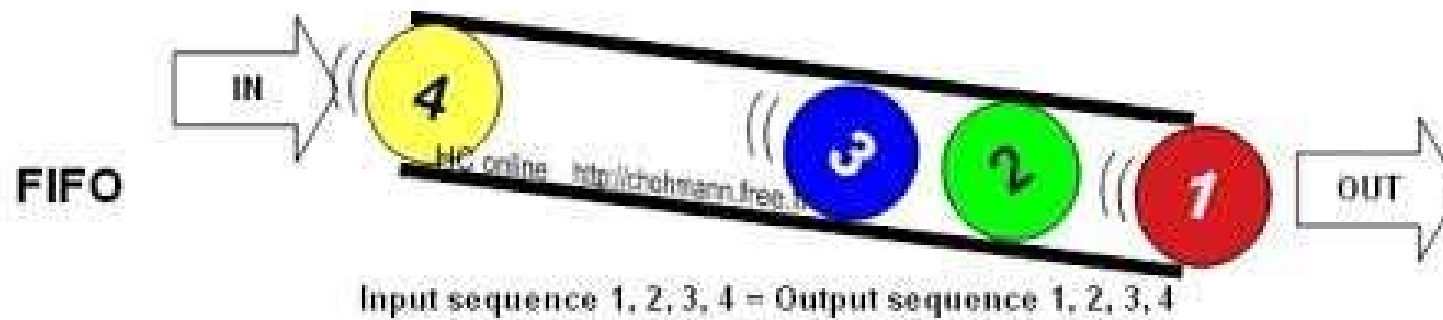
{ (A + B) * C } + D
1 2 2 2 2 1 1 1 0 0 0

((A + B)
1 2 2 2 2 1

(A + B) } - (C + D
1 1 1 1 0

Parenthesis count at various point of an input string

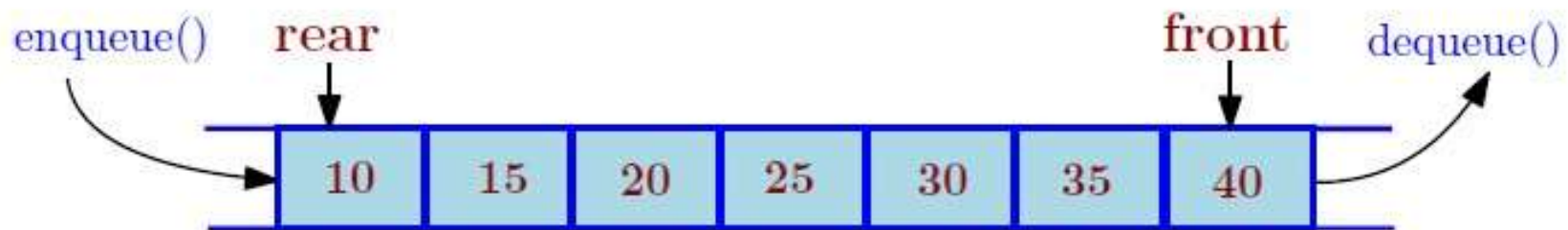
Queue



Overview

Definition: A queue is an ordered list in which all insertions take place at one end, the rear, while all deletions take place at the other end, the front.

➤ First-In-First-Out Data Structure



The Queue ADT

- ❑ The Queue ADT stores arbitrary objects
- ❑ Insertions and deletions follow the first-in first-out scheme
- ❑ Insertions are at the rear of the queue and removals are at the front of the queue
- ❑ Main queue operations:
 - **enqueue(object)**: inserts an element at the end of the queue
 - **dequeue()**: removes the element at the front of the queue

Auxiliary queue operations:

- **object front()**: returns the element at the front without removing it
 - **integer size()**: returns the number of elements stored
 - **boolean empty()**: indicates whether no elements are stored
-

Applications of Queues

❑ Direct applications

- Waiting lists
- Access to shared resources (e.g., printer)
- Multiprogramming

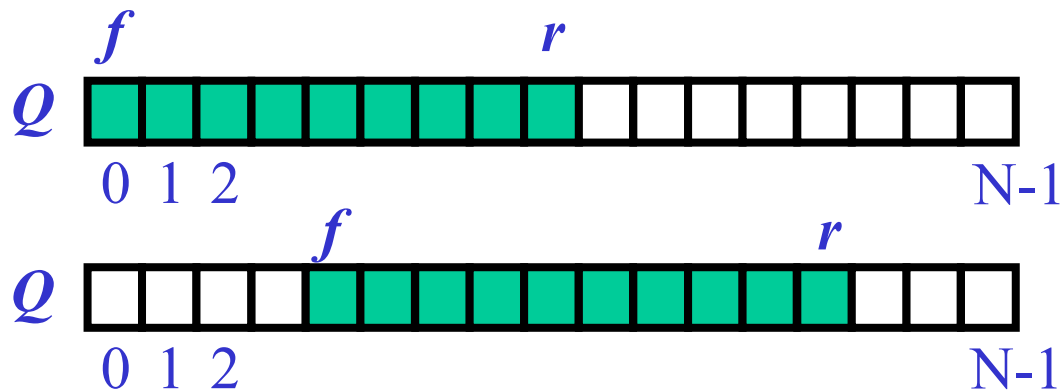
❑ Indirect applications

- Auxiliary data structure for algorithms
 - Component of other data structures
-

Array-based Queue

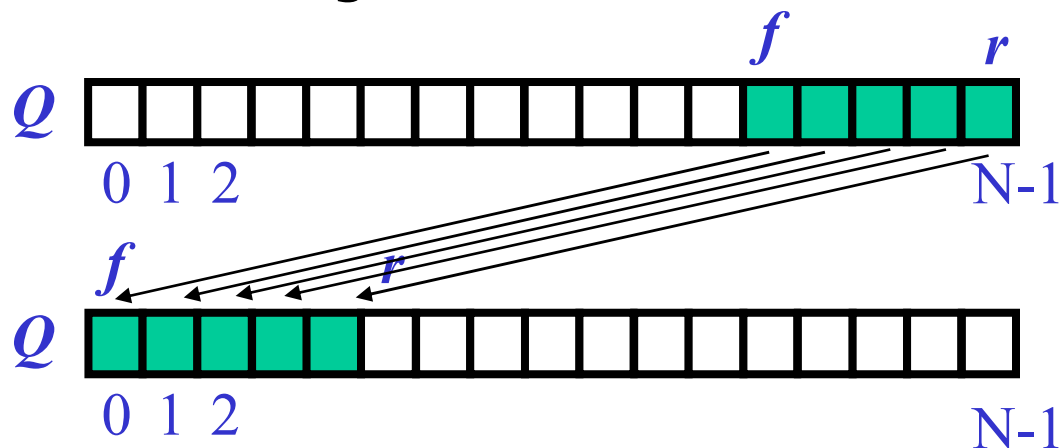
- ❑ Use an array of size N
- ❑ Two variables keep track of the front and rear
 - f : index of the front element
 - Initially set as 0
 - r : index immediately past the rear element
 - Initially set as -1
 - number of items in the queue ($r - f + 1$)

normal configuration

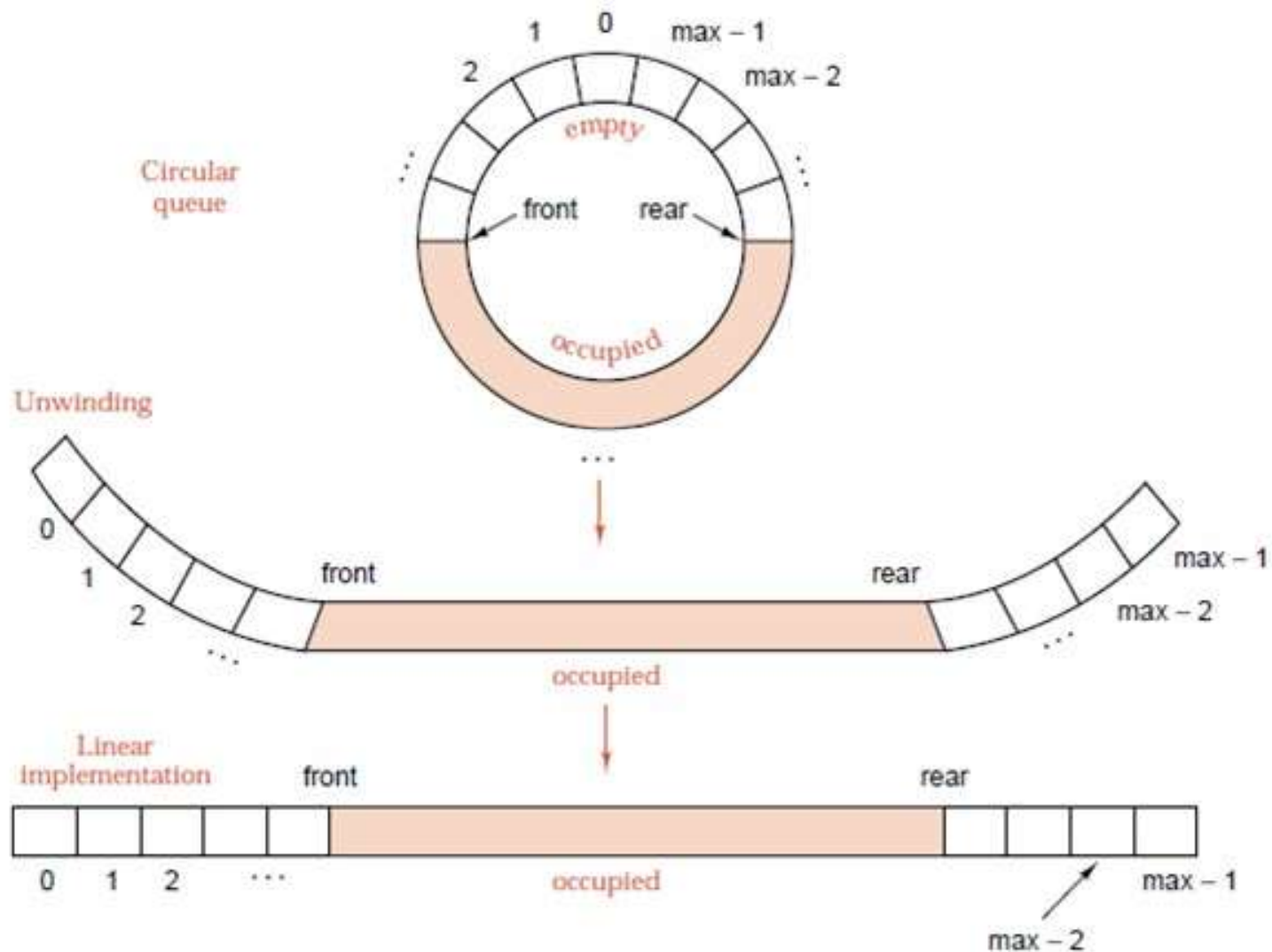


Array-based Queue

- ❑ Cons: queue will gradually move to the right and finally “queue full” is signalled even though queue is not full.
- ❑ Soln: the entire queue needs to be shifted towards left such that the first element is again at $Q(0)$ and $\text{front}=0$.
- ❑ This is very time consuming process specially, when there are many element in the queue at the time when “queue full” is signalled.

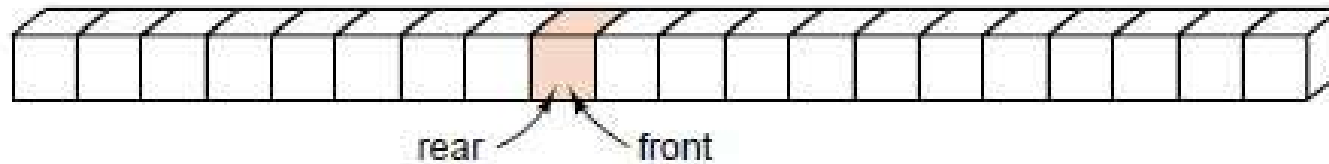


Queue in a circular Array



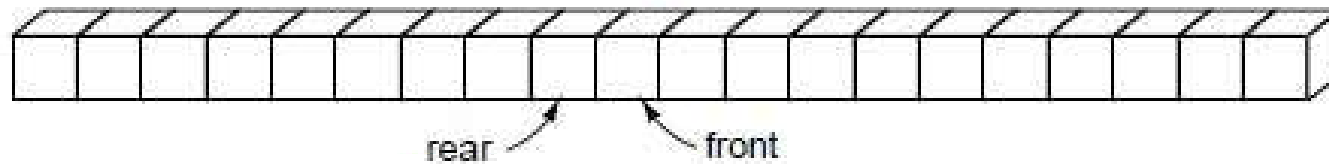
Empty and full queues

Queue
containing
one item

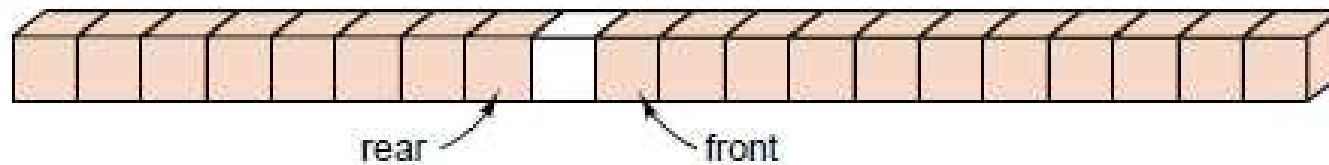


Remove the item.

Empty
queue

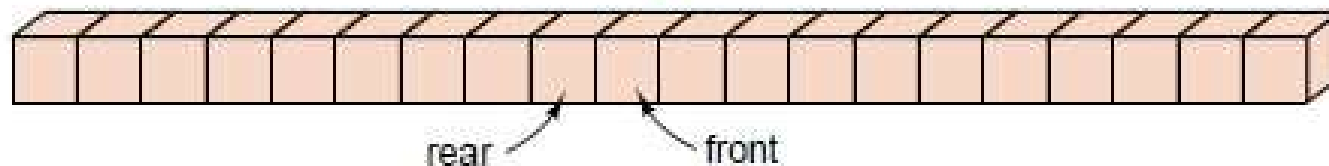


Queue
with one
empty
position



Insert an item.

Full
queue

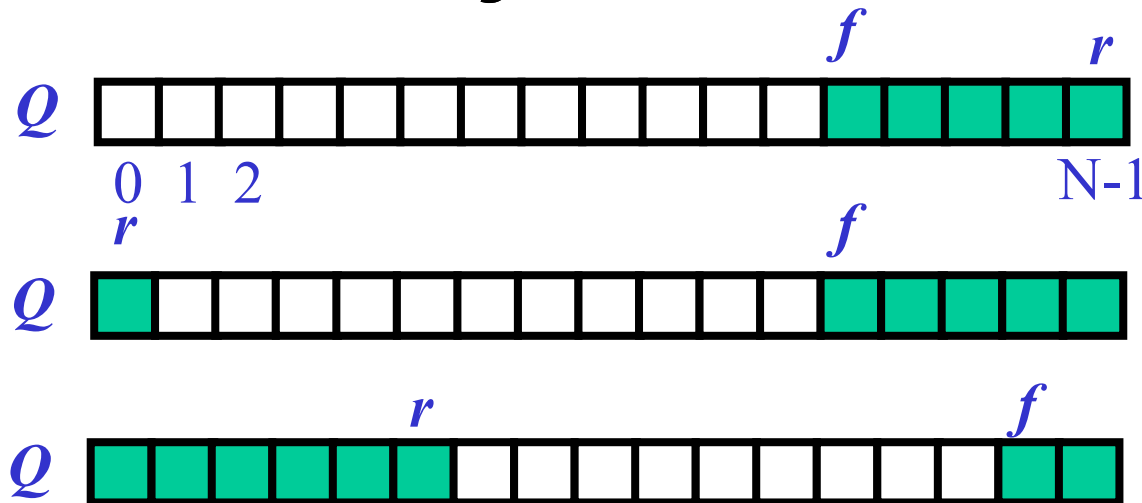


Array-based Circular Queue

❑ Use an array of size N in a circular fashion

- f : index of the front element
 - Initially set as 0
- r : index immediately past the rear element
 - Initially set as 0
- number of items in the queue ???

❑ Also called *Ring Buffer*



wrapped-around
configuration

Solution 1 (Algorithm without count)

Algorithm *size()*

?????

Algorithm *enqueue(o)*

if $r = N-1$ then

$r \leftarrow 0$

else

$r \leftarrow (r + 1)$

$r \leftarrow (r + 1) \bmod N$

if $r = f$

error *QueueFull* (revert r value)

else

$Q[r] \leftarrow o$

Algorithm *empty()*

return ($r = f$)

Algorithm *dequeue()*

if *empty()* then

error *QueueEmpty*

else

if $f = N-1$

$f \leftarrow 0$

else

$f \leftarrow (f + 1)$

$f \leftarrow (f + 1) \bmod N$

return $Q[f]$

- ❑ It sacrifice one element of the array and to allow a queue to grow only as large as one less than the size of the array

Solution 2 (Algorithm with count)

Algorithm *size()*
return *count*

Algorithm *empty()*
return (*count* = 0)

Algorithm *full()*
return (*count* = N)

Algorithm *enqueue(o)*
if *full()*
 error *QueueFull*
else
 if $r = N-1$ then
 $r \leftarrow 0$
 else
 $r \leftarrow (r + 1)$
 $r \leftarrow (r + 1) \bmod N$

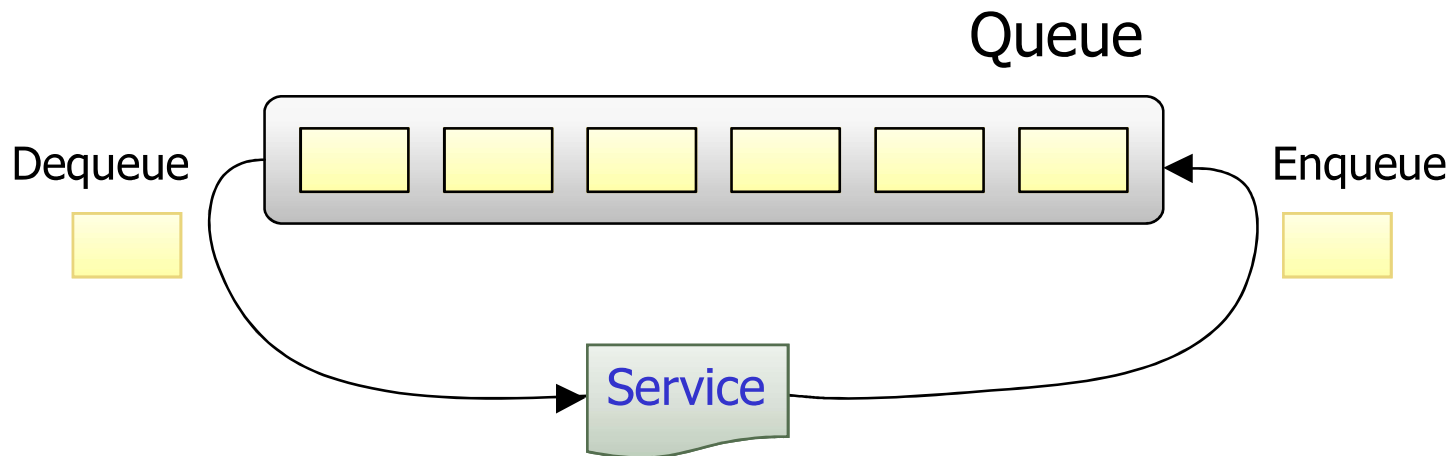
 $Q[r] \leftarrow o$
 $count \leftarrow count + 1$

Algorithm *dequeue()*
if *empty()* then
 error *QueueEmpty*
else
 if $f = N-1$
 $f \leftarrow 0$
 else
 $f \leftarrow (f + 1)$
 $f \leftarrow (f + 1) \bmod N$
 $count \leftarrow count - 1$
 return $Q[f]$

Application: Round Robin Schedulers

We can implement a round robin scheduler using a queue Q by repeatedly performing the following steps:

1. $e = Q.dequeue()$
2. Service element e
3. $Q.enqueue(e)$



Priority Queue

- ❑ It is a collection of elements stored according to their priority levels.
 - ❑ Inserting and removing elements from a queue is decided by the priority of the elements.
 - ❑ An element of higher priority is processed first.
 - ❑ Two elements of the same priority are processed on a first-come-first-served basis.
 - ❖ Example: Suppose you have a few assignments from different subjects. Which assignment will you want to do first?
 - ❖ **Later this topic will be discussed in detail.**
-